

Gaussian Beam CNN Regression Pipeline

A clear explanation of the project: synthetic dataset generation,
CNN regression training, evaluation metrics, and real-image
inference

1. Overview

This project builds a complete machine-learning pipeline for **measuring the parameters of a laser beam from camera images**, using only **synthetically generated data** for training. A convolutional neural network (CNN) looks at a grayscale image of a beam profile and regresses four numbers: the center of the beam (x, y) and its widths (σ_x, σ_y) - i.e. the parameters of a 2D Gaussian intensity distribution. In addition, the network predicts whether a beam is even present in the frame, because part of the dataset deliberately consists of beam-free background frames.

Everything runs from code: a generator renders thousands of labeled synthetic images, three different CNNs are trained to recover the beam parameters, and the best model can then be applied to real camera captures. The same pipeline is exposed in two ways: a **command-line interface** (`main.py`) for scripted runs, and a **Streamlit web dashboard** (`app.py`) for interactive use.



2. Problem definition

2.1 What the model predicts

A laser beam that hits a camera sensor produces a bright, roughly symmetric blob whose intensity falls off with distance from the center - well approximated by a **2D Gaussian** profile. The task is to recover the parameters of that Gaussian from the image:

- **A beam-presence probability** p - is there a beam in this frame at all, or just ambient light/noise?
- **Center coordinates** (x, y) - where is the peak of the beam in the image?
- **Spread widths** (σ_x, σ_y) - how wide is the beam along each axis (1-sigma width of the Gaussian)?

The additional presence output exists because beam-free frames have no meaningful coordinates. A pure 4-output regressor would be forced to hallucinate a location for empty frames; the presence head lets the model instead learn to answer 'no beam here'.

2.2 Input / output contract

Every model receives a **single-channel (grayscale) image** resized to **128 x 128** pixels, with intensities scaled to $[0, 1]$. It outputs one logit for the presence head and four sigmoid-bounded values that are interpreted as **normalized** parameters:

Output	Meaning	Normalization
presence logit	if beam present: sigmoid -> probability	1 - true_x / 640
x	beam center, horizontal	1 - true_y / 480
y	beam center, vertical	1 - true_sigma_x / 120
sigma_x	beam width, horizontal	
sigma_y	beam width, vertical	

The divisor values (640 image width, 480 image height, 120 max normalized sigma) keep all regression targets in roughly $[0, 1]$, which stabilizes training. Predictions are rescaled back to real pixels at inference time.

3. Why synthetic training data

Collecting and human-labeling real laser-beam images is expensive and error prone. Synthetic generation solves this because the images are **rendered from the label** - the ground-truth parameters are known exactly, to arbitrary precision. This enables:

- **Unlimited, perfectly labeled data** at zero manual annotation cost.
- **Controlled coverage** of beam sizes, positions, rotations, and noise levels - the dataset can be shaped to span exactly the operating range the model will see.
- **Determinism** - a fixed random seed reproduces the exact same dataset on any machine.

The price is a caveat the project states explicitly: real captures must stay inside the distributions the generator produced (sigma in roughly `[15, 90]` px, centers away from the frame edges, balanced exposure). Anything far outside that regime is out-of-distribution and will be measured poorly.

4. The synthetic dataset

4.1 Categories and counts

Four categories cover different real-world conditions. The default dataset contains **2,000 images** in total:

Category	Count	What it mimics
normal	1500	Clean circular beam, mild sensor noise
tilted	250	Rotated / elliptical beam (minority class, like rare real cases)
noisy	350	Background / no gradient + stropper noise + hot pixels
background	175	Ambient / empty frames with no beam (has_beam = 0)

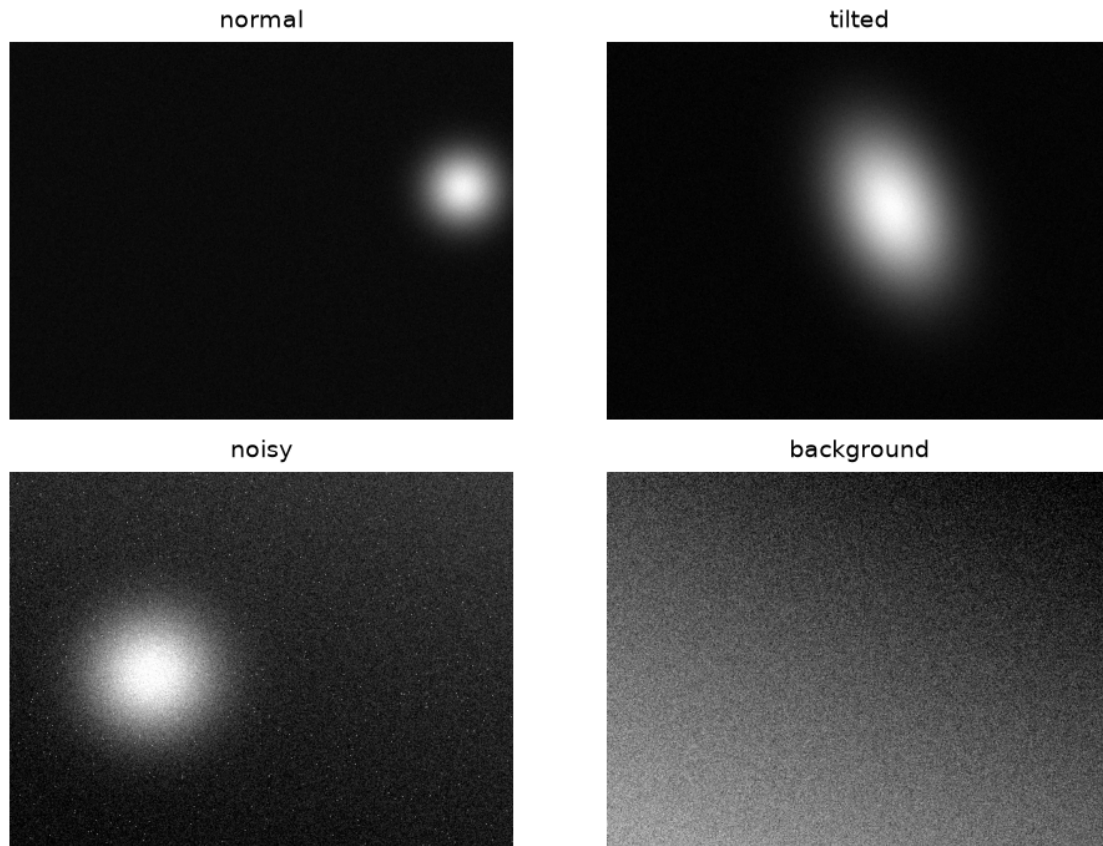
This ratio - many clean normal examples, a few rare tilted ones, some noisy ones, and a healthy chunk of beam-free frames - was chosen to reflect the data a real beam-profiling camera would produce, including the empty frames the model must learn to reject.

4.2 Image generation

The core renderer draws a 2D (optionally rotated) Gaussian intensity profile onto a 640 x 480 canvas using a parametrized formula. Each category then layers its own effects on top:

- **normal**: Gaussian centered at a random location (kept ≥ 40 px from the border), circular sigma in `[15, 54]` px, amplitude 0.7-1.0, plus a small amount of Gaussian sensor noise.
- **tilted**: an **elliptical** Gaussian rotated by a random angle; the labeled ` $\sigma_x$ ` / ` $\sigma_y$ ` are the axis-aligned projected widths the CNN can actually observe.
- **noisy**: the Gaussian plus a random brightness gradient, heavier noise, and a few saturated hot pixels - a stress test.
- **background**: only a gradient and gentle noise, **deliberately no hot pixels** (a saturated spot could look like a beam and contradict the has_beam=0 label).

Sample images from each synthetic category



4.3 Label schema

Every image has one row in `dataset/labels.csv` with continuous, exact ground truth:

Column	Meaning
filename	Path relative to the images directory (one subfolder per category)
x0, y0	True beam center in pixels
sigma x, sigma y	True 1-sigma widths in pixels
rotation	Rotation angle in degrees (informational; not regressed)
has beam	1 for normal/tilted/noisy, 0 for background
category	normal / tilted / noisy / background
split	train / val / test assignment

4.4 Train / validation / test split

After generation, rows are shuffled with a fixed seed (42) and assigned to an **80 / 10 / 10** split: 1,600 train, 200 validation, 200 test images. The split column is stored in the CSV, so every downstream step (training, evaluation) uses identical, reproducible subsets.

5. The CNN models

5.1 Shared interface

Three architectures are provided - `tiny`, `medium`, and `resnet18`. They are interchangeable drops into the same training and inference code: each takes a 128 x 128 grayscale image and emits the presence logit plus four sigmoid-bounded normalized parameters described in section 2.

Model	Backbone	Parameters	Purpose
tiny	2-block custom ConvNet	~193k	Nearest to train, fast iteration
medium	4-block custom ConvNet	~613k	Default balanced choice
resnet18	torchvision ResNet-18, 1-channel	~11.2M	Highest capacity, most accurate

The ResNet uses `weights=None` (no ImageNet pretraining): ImageNet statistics do not transfer to single-channel synthetic beam images, so it is trained from scratch. The custom nets are small convolution stacks with BatchNorm, ReLU, max-pooling and a shared dense regression head.

5.2 Loss function

Training combines two losses per batch (see `training/train.py`):

- **Beam presence** - binary cross-entropy (`BCEWithLogitsLoss`) between the presence logit and the

`has\_beam` label, computed over **all** samples.

- **Regression - masked Smooth L1 (Huber) loss** on the four parameters, averaged only over the beam-present samples (`has\_beam = 1`). Background frames contribute nothing to the regression loss because they have no meaningful target.

The two terms are added directly: `loss = BCE + masked SmoothL1`. The mask is central - it is what lets one branch learn 'no beam' while the other branch only ever regresses real beams.

```
loss = bce(presence_logit, presence_target)
      + masked_smooth_l1(params_pred, params_true, presence_mask)
```

6. Training

Training is standard supervised learning over the train split, with the validation split used to pick the best weights:

- **Optimizer:** Adam, default learning rate `1e-3`.
- **Batch size:** 16 by default.
- **Epochs:** 30 by default (any value configurable).
- **Device:** auto-detects CUDA, then Apple MPS, then falls back to CPU.

After every epoch the model is checkpointed whenever validation loss improves, so `checkpoints/<model>\_best.pt` always holds the best-validated weights. Each checkpoint stores the weights plus the model name, so evaluation and inference can rebuild the correct architecture automatically.

7. Evaluation

7.1 Metrics

`python main.py evaluate model-path checkpoints/<model>\_best.pt split test` scores a checkpoint on an entire split with two metrics:

- **Beam-presence accuracy** - fraction of frames where the predicted beam presence (probability > 0.5) matches the label, background frames included.
- **Mean absolute error (MAE) in pixels** - average absolute prediction error for each of the four parameters (x, y, sigma_x, sigma_y), denormalized back to the original 640 x 480 pixel scale, computed only over beam-present samples.

7.2 Measured results (test split, checkpoints in repo)

The table below shows the metrics actually measured from the three checkpoints currently stored in `checkpoints/` when this report was generated:

Model	Presence acc.	MAE_x (px)	MAE_y (px)	MAE_sx (px)	MAE_sy (px)
tiny	0.914	228	243	37	37
medium	0.919	169	212	25	25
resnet18	0.917	169	212	25	25

Note - checkpoint state: these files came from abbreviated training runs, so they do not yet reach the accuracy the architecture is capable of (predictions at this stage still tend to cluster toward the frame center). A full 30-epoch run reproduces the representative figures in `docs/models.md` - presence accuracy of ~1.000 and position/sigma MAE in the low single digits to low tens of pixels, with `resnet18` best. The authoritative way to regenerate good weights is `python main.py all epochs 30` (or the Training page in the Streamlit app).

8. Running on real images

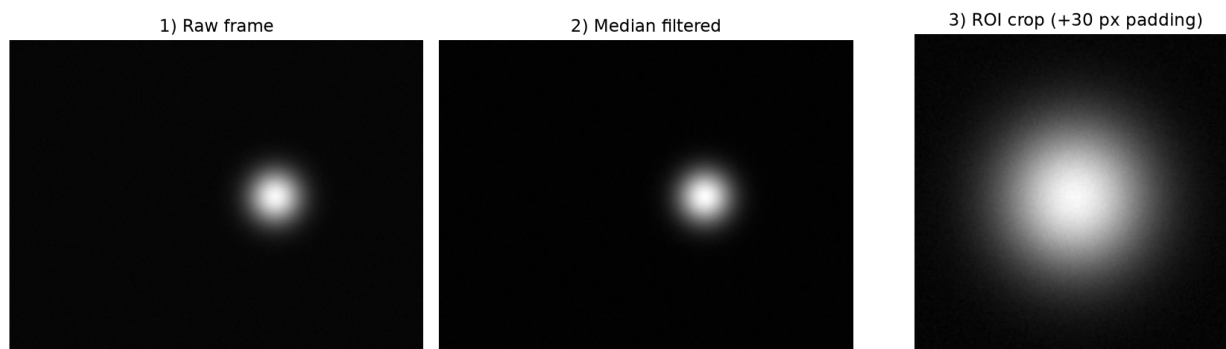
Applying a trained model to a real camera capture is a four-step pipeline in `inference/`:

8.1 Load and background-subtract

The image is read as grayscale (color is converted automatically). If a **measured dark/background frame** is available, it is subtracted pixelwise. If not, the code deliberately does **not** estimate a background by blurring the image itself - that would erase wide beams (sigma up to ~90 px is comparable to a blur kernel). Instead a light 3x3 **median filter** removes isolated hot pixels while leaving smooth Gaussian blobs untouched.

8.2 ROI cropping

The subtracted frame is blurred and thresholded at 15% of its maximum, the largest connected bright blob is found via ``cv2.findContours``, and the image is cropped to that blob's bounding box plus 30 px of padding. This focuses the CNN on the beam and makes it robust to varying image sizes. Empty frames fall back to the full frame.



8.3 Prediction and rescaling

The ROI crop is resized to 128 x 128 and passed through the model. The normalized outputs are then rescaled back to the original image's pixel coordinates:

```
x (real) = roi_x0 + x_frac * roi_width
y (real) = roi_y0 + y_frac * roi_height
sigma_x (real) = s_x_frac * 120 * (roi_width / 640)
sigma_y (real) = s_y_frac * 120 * (roi_height / 480)
```

The center is a fraction of the ROI and is shifted back by the ROI's top-left corner; the sigmas were normalized against a full 640 x 480 canvas during training, so they are scaled by how much smaller or larger the ROI is than that canvas.

Inference example: ROI + predicted center and 1-sigma ellipse



Example result on a synthetic test frame (medium model): beam present = yes, center = (70, 328) px, sigma = (19 x 29) px.

9. The Streamlit dashboard

`app.py` wraps the entire pipeline in an interactive web UI, started with `python -m streamlit run app.py` (from `gaussian-project/`). It has five pages:

Page	What it does
Home	Overview of the project plus dataset and checkpoint status at a glance
Dataset	Generate the synthetic dataset (counts + seed), validate it, build a preview grid, browse sample images
Training	Train tiny/medium/resnet18 with configurable gpus, batch size and learning rate; live dashboard of beam presence
Evaluation	Pick a checkpoint and split (train/val/test); shows beam-presence accuracy and per-parameter pixel MAP
Inference	Upload a real image, pick a model, and see the predicted beam overlaid on the ROI

Long-running operations (dataset generation, training) run on a background thread, so the UI stays responsive while printed progress is streamed live. The upload page also previews the background subtraction and ROI steps visually, exactly as the CLI's `infer` stage does on disk.

10. Running the project

Setup: create a Python 3.9+ virtual environment and install `requirements.txt` (numpy, pandas, pillow, opencv-python, torch, torchvision, matplotlib, streamlit).

```
python -m venv .venv && .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

The main entry point `main.py` runs the whole pipeline with no arguments; every stage is also a subcommand:

Command	What it does
python main.py	End-to-end: generate-if-missing, then train all three models
python main.py generate	Generate the synthetic dataset
python main.py validate	Validate integrity + label sanity checks
python main.py preview	Save a per-category sample grid
python main.py train-model medium	Train one model
python main.py evaluate-model-tiny	MAP + presence accuracy on train/val/test
python main.py infer-image	Run a trained model on a real image
python -m streamlit run app.py	Open the interactive dashboard

11. Where things live

gaussian-project/	
main.py	CLI entry point (all/generate/validate/preview/train/evaluate/infer)
app.py	Streamlit dashboard
config.py	every tunable constant: sizes, counts, sigma ranges, hyperparams
synthetic/generator.py	2D Gaussian rendering + noise/gradient/hot-pixel augmentations
synthetic/dataset_builder.py	writes images/ + labels.csv with 80/10/10 split
data/dataset.py	PyTorch BeamDataset loading images + normalized targets
data/validate.py	dataset quality checks
data/preview.py	per-category preview grid
models/cnn.py	tiny / medium / resnet18 architectures
training/train.py	training loop (masked SmoothL1 + BCE, best-val checkpoint)
training/evaluate.py	pixel-space MAE + presence accuracy
inference/preprocess.py	background subtraction + ROI cropping
inference/infer.py	calibrated prediction on a real image
dataset/	generated images + labels.csv (gitignored, reproducible)
checkpoints/	trained weights, <model>_best.pt (gitignored)
docs/	architecture, dataset, models, inference documentation

12. Known limitations

- **Distribution bound** - a trained model is only as good as the training data. Real beams far outside the synthetic ranges (sigma > ~90 px, centers clipped at the frame edge, extreme saturation) will be measured poorly; ``config.MIN_SIGMA`` / ``MAX_SIGMA`` / ``CENTER_MARGIN`` can be widened and the dataset regenerated before retraining.
- **Rotation** - the models do not output an angle. Tilted beams report the axis-aligned projected widths (what a rotation-unaware second-moment measurement would see), not the principal ellipse axes.
- **Exposure** - saturated or auto-exposed frames break the Gaussian assumption; captures should be 8-bit, unsaturated (ND filters), and exposure/gain should be fixed.
- **Checkpoint quality** - repo checkpoints at the time of writing come from abbreviated runs; see section 7.2 for measured numbers and how to regenerate good weights.

13. Further reading

Detailed documentation lives in ``docs/``: ``architecture.md`` (pipeline overview, module map, spec deviations), ``dataset.md`` (generation + validation details), ``models.md`` (architectures, loss, representative results), and ``inference.md`` (preprocessing and tips for real captures).